

Missile Command Repair Lab - Evidence Report

Prepared 7 October 2026. Application: Lab-4/game.py. Original/before version: root game.py.

Evidence status

Implementation and deterministic checks are complete. Real graphical gameplay, screenshots, before/after videos, the individual repository URL, and the complete chat-history URL are NOT supplied by this report.

Pygame was unavailable in this environment. An explicit installation attempt against PyPI failed with: "Could not find a version that satisfies the requirement pygame" / "No matching distribution found for pygame".

Consequently no genuine game-window screenshot or gameplay video was captured. There are no simulated screenshots or fabricated links.

The tests execute the actual application methods using a small test-only vector double when Pygame cannot be imported. Font/screen/drawing spies confirm draw calls, message positions, and colors; they do not establish visual appearance in a real window. Run the suite with Pygame on your own machine and visually review normal gameplay before submission.

Changes and commits

Change	Commit	Actual implementation
Original baseline	b4318a9	Original game.py and supplied assignment README.
Task 1	aca9367	Filter alive batteries with positive ammo, select the nearest eligible battery, return None if none; guard launch and spend exactly one ammo.
Task 2	422c9c6	Clamp progress; piecewise linear white/yellow/orange/red interpolation with rounded integer RGB.
Task 3	516b72e	Per-city two-second destruction timer; draw CITY DESTROYED! near city; decrement timers even after game-over; reset creates fresh timers.
Task 4	f549159	city_repair_threshold returns 2000; uses existing repairs_awarded and game-over guard.
Review fixes + regression suite	2c901de	Cap interceptor movement at the target; consume each crossed milestone individually using the existing counter; add separate deterministic tests.
Submission documentation	See Git history	Separate documentation commit containing this report, README, prompt appendix, PDF, and raw logs. Resolve its hash with the command below.

```
git log -1 --format='%h %s' -- Lab-4/EVIDENCE.md
git log --oneline -- Lab-4
```

A report cannot embed the hash of the same commit containing itself. The exact documentation hash is provided in the final chat response and is available in Git history. No commits have been pushed to the course or an individual repository by the assistant. The configured author was already Codex <codex@openai.com>; no student identity was invented or assigned.

Reproducible issues found in review

1. Interceptor arrival: at the normal maximum dt of 0.05 seconds, movement is 21 pixels. An interceptor whose target is 31 pixels away can alternate between positions with remaining distances of 10 and 11 pixels and never enter the old six-pixel arrival tolerance. Separately, a target five pixels away detonated immediately even when the frame allowed only 0.42 pixels of movement. The fix compares distance with this frame's movement, snaps to the exact target on arrival, and never moves past it. 2. Skipped repairs: a score jump to 6000 with three destroyed cities updated repairs_awarded directly to 3 but restored only one city. The fix increments the existing counter once per crossed milestone and attempts one restoration per milestone. Milestones with no destroyed city are still consumed, so later destruction cannot reuse an old award. The game-over return still prevents any revival after losing.

The initial combined suite ran 24 tests: 20 passed, four failed. Three failures reproduced arrival/early-detonation behavior; one reproduced skipped milestones. See evidence/review-before-fixes.txt. After the minimal fixes, the same 24 tests all passed. See evidence/verification-results.txt.

Actual final verification results

Command: python3 Lab-4/tests/test_game.py

Actual run: 24 tests, OK, exit code 0. Random seed: 33 in each test setup. No GUI or video evidence is claimed.

Area	Checks performed	Result
Launch and ammunition	Closest available, depleted, destroyed, mixed unavailable, empty list; last ammo then fallback; correct origin/target; exactly one ammo consumed; ground and lose-state guards.	PASS
Explosion colors	Inputs -2, 0, 1/3, 0.5, 2/3, 1, 2; 1001 bounded integer RGB samples; neighboring sample differences at most one channel unit.	PASS
Interceptor arrival	31-pixel target at 0.05 dt; five-pixel target at 0.001 dt; already at target; Game.update explosion created at exact target.	PASS
Explosion timing/radius	Radius starts at 0, reaches 45 halfway through 1.2 seconds, returns to 0 and expires; draw uses the midpoint color and radius.	PASS
Collision	Two heads inside radius removed and award 25 each; outside head survives; score not awarded twice; current radius used instead of maximum. Existing strict less-than boundary semantics preserved.	PASS
Missile impacts	Missile advances to its target; city and battery destroyed on impact; impact explosion radius 30; repeat city hit does not retrigger callback.	PASS
City feedback	Message draw call near the city, two-second duration, repeat-hit/battery exclusion, expiry during play and loss; final-city message and game-over draw calls both present.	PASS
Wave progression	Surviving-city and remaining-ammo bonuses unchanged; wave increments; next spawn count and battery refill; destroyed city remains destroyed until a repair.	PASS
Repair	No repair at 1999; one at 2000/4000/6000; same milestone not repeated; no destroyed city causes no error or banked reward; multiple crossed milestones; actual +25 interception crosses milestone and next update repairs.	PASS
Game over	Final city loss sets lose; launch disabled; score milestones do not revive lost game; feedback expires while game-over remains.	PASS
Reset	Score, wave, state, ammo, cities, timers, active objects, spawn count, and repairs_awarded reset. R event binding unchanged.	PASS
Static preservation	Application and tests compile; root game.py byte-identical to baseline; Task 1 methods unchanged; production scoring, impact guards, missile movement, explosion timing/radius and main event loop unchanged.	PASS

Selected exact color results: progress 0 = (255,255,255); progress 0.5 = (255,192,0); progress 1 = (255,0,0). Below/above-range inputs clamp to the start/end respectively.

Files and scope

- Lab-4/game.py: final application.
- Lab-4/tests/test_game.py: deterministic verification only; no production import. Forced scores, destruction, and stopped spawning exist only in this test script.
- Lab-4/README.md: setup, controls, task summary, submission instructions, recording plan.
- Lab-4/PROMPTS.md: exact three user prompts available in this conversation.
- Lab-4/EVIDENCE.pdf: PDF copy of this report plus the prompt appendix.
- Lab-4/evidence/: raw before-fix and final test logs.

Missing user-provided details and evidence

Item	Placeholder / required action
Student name	[MISSING: verify and fill your name]
SRN and section	[MISSING: verify and fill your SRN and section]
Individual repository URL	[MISSING: YOUR_INDIVIDUAL_REPOSITORY_URL]
Before-video URL	[MISSING: YOUR_BEFORE_VIDEO_URL]
After-video URL	[MISSING: YOUR_AFTER_VIDEO_URL]
Complete chat-history URL	[MISSING: YOUR_CHAT_HISTORY_URL]
Genuine screenshots	[MISSING: capture from real original/final game windows if required]
Submission portal/deadline and complete email	[MISSING: check the professor's actual instructions]

The course origin https://github.com/SETAPESU26/33_missileCommand is the supplied starting repository, not a verified individual submission URL. Replace origin with your own URL before pushing. Do not invent evidence or claim these placeholders are complete.

Recording and chat preservation

Record normal gameplay with the original root game.py for before and Lab-4/game.py for after. Rehearse depletion, fallback, explosion colors, city destruction, and milestone repair. Record a longer genuine session and trim a continuous 10-second interval. Exact scheduling of all four features is not guaranteed; use supplementary genuine clips when needed. Never force production scores or use test setups as normal-play evidence. Detailed recording steps are in README.md.

Keep the complete conversation, not just the three prompts. After the final response, create/refresh a shared chat link if available and verify recipient access. Save/export or print the full conversation as backup, including assistant responses and available tool activity. The prompt appendix is a transcript of user requests only and is not proof of complete chat preservation.

Exact user prompts from this conversation

The three blocks below preserve the three user implementation prompts verbatim, including the literal backslash in Prompt 3. These are user prompts only. Preserve the complete chat separately, including assistant responses and available tool activity. No complete chat-history URL has been provided.

Prompt 1

Work directly in this repository to complete my Missile Command Repair Lab. Implement changes, rather than only explaining them.

Read README.md, game.py, and any applicable repository instructions. The submission email requires the updated application under Lab-4 and a separate Git commit for each of the four tasks.

Create Lab-4/game.py from the original game.py. Preserve the original root game.py so I retain the before version. Make all application changes in Lab-4/game.py.

Complete Task 1:

- nearest_battery(target) must consider only batteries that are alive and have ammo greater than zero.
- Return the nearest eligible battery.
- Return None if no eligible batteries exist.
- Update launch() to handle None without crashing or consuming ammunition.
- Preserve the existing targeting and gameplay rules.

Verify selection when the closest battery is depleted, destroyed, available, and when every battery is unavailable. Check that a successful launch consumes exactly one ammunition unit.

Create a separate commit for Task 1. Do not implement Tasks 2–4 yet. If Git commits are blocked by missing author configuration, report that and provide the exact commands without inventing my identity.

Finish with a concise explanation of the change, actual verification results, the commit hash if created, and the terminal command to run Lab-4/game.py.

Prompt 2

Continue working in Lab-4/game.py and preserve the verified Task 1 fix.

Complete Task 2:

Implement explosion_color(progress) with a smooth transition from white-hot through yellow/orange to red as the explosion ages. Clamp progress to 0–1 and return integer RGB components between 0 and 255. Preserve explosion timing, radius, and collision behaviour. Verify the start, midpoint, end, and out-of-range inputs. Create a separate Task 2 commit before beginning Task 3.

Complete Task 3:

Implement on_city_destroyed(city) so destroying a previously standing city produces visible, temporary feedback in the game, such as “CITY DESTROYED!” near that city. Use the existing callback signature and a simple timer-based approach. Avoid external assets and unnecessary global state. Ensure the feedback is triggered only for a newly destroyed city, expires correctly, and resets when R restarts the game. Make sure the feedback is still visible if the destroyed city was the last surviving city. Verify that hitting an already destroyed city or hitting a battery does not trigger city-destruction feedback. Create a separate Task 3 commit.

Keep the implementation small and readable. Do not implement Task 4 yet. Finish with actual verification results, commit hashes if created, and instructions for observing both features.

Prompt 3

Complete the remaining work without requiring another AI prompt.

Complete Task 4 in Lab-4/game.py:

- Implement `city_repair_threshold()` to return 2000.
 - Use the existing `repairs_awarded` bookkeeping.
 - Verify that reaching each new 2000-point milestone restores one destroyed city when one is available.
 - Verify that the same milestone does not repeatedly restore cities, that having no destroyed cities causes no error, and that R resets the repair bookkeeping.
 - Preserve game-over behaviour.\
- Create a separate Task 4 commit.

Review and verify all four tasks together. Check launching, ammunition use, interceptor arrival, explosion collisions, missile impacts, wave progression, city repair, game over, and reset. If you discover a reproducible issue affecting the README's expected behaviour, make a minimal fix, explain it, and put it in an additional separate commit.

Use deterministic checks where possible. Any helper that forces scores or destruction for testing must be separate from normal gameplay. Do not add fake gameplay evidence or alter production scoring to make the demonstration easier.

Prepare submission documentation under Lab-4:

- README with setup, controls, and all four implemented tasks.
- An evidence report containing the changes, actual test results, commit information, and genuine screenshots if the environment supports capturing them.
- Generate a PDF version if supported. Clearly mark any missing user-provided information.
- Include the three exact prompts from this conversation if available, and explain how I should preserve the complete chat history.
- Provide labelled placeholders for my individual repository URL, before-video URL, after-video URL, and chat-history URL. Do not invent links or claim missing evidence is complete.

Commit documentation separately. Finish with the exact run and Git push commands, a practical 10-second after-video recording plan, and a checklist of anything I must manually complete before submission.